PROJECT REPORT

HISTORICAL AIRFARE ROUTE PROFILES THROUGH REGIONAL PRICE BEHAVIOUR CLUSTERING (AEROCLUSTER)

COMPUTER SCIENCE AND ENGINEERING

Course: INT396 (Unsupervised Learning) | Problem Statement ID: SIH26056

Submitted By:

Pudi Sai Charan Teja (12507339)

Roll No: RK3P24XCB35 | Section: K3P24XC

Under the Guidance of:

Hitesh Kumar

Course Faculty — INT396 (Unsupervised Learning)



School of Computing & Artificial Intelligence

Lovely Professional University, Phagwara

Academic Session: 2026-2027

DECLARATION

I hereby declare that the project report entitled “Historical Airfare Route Profiles Through Regional Price Behaviour Clustering (AeroCluster)” submitted in partial fulfillment of the requirements for the course **INT396 (Unsupervised Learning)** under Problem Statement ID **SIH26056** in the Bachelor of Technology in Computer Science and Engineering program at Lovely Professional University, Phagwara, is an authentic record of my own independent work carried out under the guidance of **Hitesh Kumar**, Course Faculty for INT396 (Unsupervised Learning). The matter embodied in this report is original and has not been submitted by me for the award of any other degree, diploma, or evaluation.

(Signature of the Candidate)

Pudi Sai Charan Teja

Registration Number: 12507339

Roll Number: RK3P24XCB35 | Section: K3P24XC

CERTIFICATE

This is to certify that the project report entitled “Historical Airfare Route Profiles Through Regional Price Behaviour Clustering (AeroCluster)” is a bonafide record of the independent academic project work carried out by **Pudi Sai Charan Teja (12507339)** under the supervision and guidance of **Hitesh Kumar**, Course Faculty, in partial fulfillment of the requirements for **INT396 (Unsupervised Learning)** in the Bachelor of Technology in Computer Science and Engineering program.

Hitesh Kumar

Course Faculty — INT396 (Unsupervised Learning)

(Project Guide / Course Instructor)

School of Computing & Artificial Intelligence

Lovely Professional University, Phagwara

ACKNOWLEDGEMENT

I would like to express my profound gratitude to my course instructor and guide, **Hitesh Kumar**, for continuous academic support, constructive technical feedback, and encouragement throughout the development of this capstone project for **INT396 (Unsupervised Learning)**. Theoretical and practical guidance regarding feature scaling, distance metric behavior, cluster stability validation, and principal component eigendecomposition was instrumental in shaping the analytical pipeline of AeroCluster.

I also extend sincere thanks to the Head of the Department and the faculty members of the School of Computing & Artificial Intelligence at Lovely Professional University for providing the structured curriculum, laboratory infrastructure, and academic environment conducive to independent research and full-stack software deployment. Finally, I thank my family and peers for their steady encouragement during the completion of this academic work.

ABSTRACT

Domestic civil aviation pricing is governed by opaque automated yield management systems that continuously adjust ticket fares according to seat inventory scarcity, booking horizon, itinerary duration, stop count, carrier tier, and cabin configuration. Within the final 96 hours preceding flight departure, revenue management multipliers escalate Economy fares aggressively—frequently reaching +70% to +120% above fair historical route medians—while ultra-low headline deals often conceal exhausting 14-to-24 hour multi-stop layovers. Without ground-truth cluster labels, ordinary passengers and aviation analysts lack quantitative benchmarks capable of separating genuine early-bird bargains from algorithmic surge gouging or inconvenient itinerary compromises.

To address Problem Statement **SIH26056**, this project engineers **AeroCluster**, an end-to-end unsupervised machine learning pipeline and interactive production web application trained over **300,153** historical Indian domestic flight bookings (`Clean_Dataset.csv`). Mixed continuous features (`price`, `duration`, `days_left`) and categorical corridor attributes (`airline`, `source_city`, `destination_city`, `stops`, `departure_time`, `class`) are transformed using a leakage-free scikit-learn `ColumnTransformer` pairing `StandardScaler` with `OneHotEncoder(drop='first')` into a normalized 26-dimensional orthogonal vector space. The system sweeps K-Means++ across $k \in [2, 10]$, evaluates Euclidean (L_2) versus Manhattan (L_1) distance geometries, cross-validates structural stability via Agglomerative Ward linkage (ARI = 0.6522, NMI = 0.6405), isolates predatory surge gouging (≥ ₹35,000) and flash-sale glitches (≤ ₹3,000) via DBSCAN (ε = 1.65, MinPts = 15; 3.80% noise), and performs 3D PCA eigendecomposition explaining 54.81% cumulative variance.

Architected across three decoupled tiers—a serialized Python ML core (`src/`), a typed FastAPI REST backend (`api/`) with Pydantic v2 schema bounds, and a React 19 + Vite 8 frontend (`frontend/`)—AeroCluster serves both a zero-jargon Consumer Verdict & Printable PDF Valuation Certificate portal and an Analyst Laboratory featuring WebGL 3D PCA clouds and a 6-hub domestic SVG network map. Containerized via multi-stage Docker on Render Cloud (`aerocluster.onrender.com`) and verified with 7/7 passing `pytest` integration tests, the system demonstrates complete mastery of INT396 Course Outcomes CO2, CO3, and CO5.

Contents

ABSTRACT **5**

CHAPTER 1: INTRODUCTION **7**

1.1 Introduction to Dynamic Airfare Pricing 7
1.2 Background of the Study 7
1.3 Problem Statement (SIH26056) 7
1.4 Objectives and Need 8
1.5 Scope and Proposed System Overview 8
1.6 System Requirements Specification (SRS) 8

CHAPTER 2: PROJECT OVERVIEW, KEY FEATURES & LITERATURE REVIEW **9**

2.1 Dual-Mode Web Application Features and Working 9
2.2 Evolution of Dynamic Airfare Analysis 9
2.3 Clustering, Distance Geometry & Density Modeling 9
2.4 Gap Analysis & Research Contribution 10

CHAPTER 3: TOOLS, TECHNOLOGIES & SYSTEM ARCHITECTURE **11**

3.1 Tools, Technologies, and Frameworks Used 11
3.2 Three-Tier System Architecture 11
3.3 System Architecture & Execution Flowchart 12
3.4 Data Modeling & REST API Design 12

CHAPTER 4: METHODOLOGY, ALGORITHMS & MATHEMATICAL IMPLEMENTATION **14**

4.1 Dataset Specifications, Cleaning & IQR Diagnostics 14
4.2 Unbiased Preprocessing Pipeline (CO2 Foundation) 14
4.3 K-Means++ Partitioning & Internal Validation Metrics 14
4.4 Distance Sensitivity Analysis: Euclidean vs. Manhattan 15
4.5 Hierarchical Validation: Ward's Linkage, ARI & NMI 15
4.6 DBSCAN Density Anomaly Isolation 16
4.7 Principal Component Analysis (3D PCA) 16

CHAPTER 5: IMPLEMENTATION, DEPLOYMENT & SECURITY ANALYSIS **17**

5.1 Modular ML Pipeline & Serialization (src/) 17
5.2 FastAPI Inference & Centroid Proximity Scoring (api/) 17
5.3 React 19 Frontend & Isolated Print Certificate Engine 17
5.4 Multi-Stage Docker & Cloud Deployment 17
5.5 Performance Engineering & Security Considerations 17

CHAPTER 6: RESULTS, TESTING & LIVE SYSTEM SCREENSHOTS **18**

6.1 K-Means Sweep Quantitative Results (k = 2..10) 18
6.2 Cross-Algorithm Comparison & DBSCAN Anomaly Isolation 19
6.3 De-Standardized Cluster Centroids (k = 4) 20
6.4 Automated Test Suite Verification (pytest tests/ -v) 21
6.5 Live Production Web System Screenshots 21

CHAPTER 7: INDIVIDUAL CONTRIBUTIONS STATEMENT **24**

CHAPTER 8: SOCIETAL IMPACT, CONCLUSION & FUTURE SCOPE **25**

8.1 Practical Applications & Potential Societal Impact 25
8.2 Conclusion 25
8.3 Limitations & Ethical Considerations 25
8.4 Future Scope 25

REFERENCES **26**

CHAPTER 1: INTRODUCTION

1.1 Introduction to Dynamic Airfare Pricing

Airline revenue management operates on the principle of perishable inventory pricing: once a scheduled aircraft departs, any unsold seat generates zero marginal revenue, whereas constrained late demand permits carriers to capture higher willingness-to-pay from urgent corporate flyers. Consequently, automated pricing engines recalculate domestic airfares multiple times daily across lead-time windows, transit stop configurations, carrier service tiers, and departure slots.

Across India's six primary metropolitan air corridors (Delhi, Mumbai, Bengaluru, Hyderabad, Chennai, and Kolkata), identical city pairs display extreme fare dispersion spanning ₹1,105 to ₹123,071. Because conventional travel aggregators present sorted lists purely by headline fare without explaining structural cluster norms, everyday passengers struggle to differentiate fair market quotes from opportunistic surge multipliers or tiring layover compromises.

1.2 Background of the Study

Exploratory investigation across 300,153 historical domestic bookings uncovers two recurring market opacity patterns affecting consumer decision-making:

1. **The 96-Hour Algorithmic Surge Trap:** Within four days prior to departure, automated seat-bucket rationing triggers severe upward price convexity, routinely elevating Economy tickets +70% to +120% above baseline corridor medians.
2. **The Layover Discount Illusion:** Deceptively low promotional fares often conceal 14-to-24 hour multi-leg layovers via secondary hubs. Unsupervised feature clustering explicitly separates brief trunk corridors from lengthy connecting itineraries.

1.3 Problem Statement (SIH26056)

Given an unlabeled historical domestic airfare dataset $D = \{\mathbf{x}_i\}_{i=1}^{300,153}$ containing continuous metrics (`price`, `duration`, `days_left`) and categorical metadata (`airline`, `source_city`, `destination_city`, `stops`, `departure_time`, `class`), Problem Statement **SIH26056** challenges us to construct a reproducible unsupervised machine learning system that:

1. Prevents large INR fare scales from drowning out duration and lead-time variations via standardized orthogonal feature encoding.
2. Identifies optimal route pricing clusters k^* without supervision by reconciling Elbow Inertia curvature, Silhouette separation, and Davies-Bouldin separation scores.
3. Benchmarks distance metrics (Euclidean L_2 vs. Manhattan L_1), validates partition stability against Ward hierarchical agglomeration (ARI/NMI), and isolates surge outliers via DBSCAN.
4. Translates mathematical centroids back into INR, hours, and booking lead days, serving real-time passenger deal verdicts and analyst visuals through a live cloud web portal.

1.4 Objectives and Need

- **CO2 Foundation (Scaling & Partitioning):** Engineer a serialized `ColumnTransformer` combining `StandardScaler` and `OneHotEncoder(drop='first')` to construct an orthogonal 26-dimensional space; execute K-Means++ across $k \in [2, 10]$ and isolate low-density anomalies with DBSCAN.
- **CO3 Metric Sensitivity & Hierarchical Concordance:** Quantify divergence between Euclidean (L_2) and Manhattan (L_1) distance geometries on mixed binary-continuous vectors and verify non-centroid stability via Ward linkage ARI and NMI.
- **CO5 Dimensionality Reduction & Cloud Operationalization:** Compute 3D PCA projections capturing 54.81% variance, interpret component loadings, and ship a Dockerized FastAPI + React 19 production portal on Render Cloud.

1.5 Scope and Proposed System Overview

AeroCluster spans the six largest Indian domestic aviation hubs and six national carriers across Economy and Business cabins. Offline scripts ingest the dataset, persist serialized `joblib` artifacts, and export seven 300-DPI diagnostic figures. The production portal features two integrated workflows: **Consumer Intelligence Mode** delivers instant ticket deal ratings (/10), fair-corridor savings deltas, 50-day SVG price trajectories, and isolated single-page PDF Valuation Certificates; **Analyst Lab Mode** exposes cluster validation plots, DBSCAN surge diagnostics, a 6-hub interactive route corridor SVG map, and an interactive WebGL 3D PCA explorer.

1.6 System Requirements Specification (SRS)

Table 1.1: System Requirements Specification Summary (SRS)

SRS Category	Specification & Operational Threshold
Hardware Requirements	Intel Core i3 / AMD Ryzen 3 (x86_64 or ARM64); 4 GB RAM minimum (8 GB recommended for training); 500 MB free disk space.
Software Platform	Python 3.13+, Node.js 20+, Docker; FastAPI, scikit-learn, SciPy, Pandas; React 19, Vite 8, Tailwind CSS v4, Plotly.js.
Functional Scope	Ingest and clean 300,153 records (~25 MB uncompressed CSV); build 26D orthogonal pipeline; serve live inference via `POST /api/v1/predict/route-cluster`; generate printable certificates.
Non-Functional Targets	Live sub-25 ms inference latency; sub-5 ms PCA rendering via `sessionStorage`; Pydantic v2 validation guardrails; 99.9% uptime on Render Cloud.

CHAPTER 2: PROJECT OVERVIEW, KEY FEATURES & LITERATURE REVIEW

2.1 Dual-Mode Web Application Features and Working

To serve diverse audiences seamlessly—both price-conscious travelers seeking zero-jargon booking verdicts and data science evaluators inspecting clustering metrics—AeroCluster separates navigation into distinct functional modes documented in Table 2.1.

Table 2.1: Functional Modes & Interactive Modules of AeroCluster Web Portal

System Mode	Frontend Components	Implemented Capabilities & User Workflow
Consumer Intelligence Mode	`HomePage.jsx`, `AssessmentReportModal.jsx`, `SurgeTrajectoryChart.jsx`	Interactive departure calendar picker, airline/route selection, instant AI Deal Verdict & /10 Value Rating, fair benchmark savings delta, interactive 50-day SVG Surge Curve showing 30–50d sweet spot vs. 1–7d surge cliff, and Downloadable Valuation Certificate modal with single-page PDF export.
Analyst Lab Mode	`DiagnosticsPage.jsx`, `PCASpacePage.jsx`, `ArchetypesPage.jsx`	Multi-*k* Elbow WCSS, Silhouette & Davies-Bouldin validation plots; Scree plot; Euclidean vs. Manhattan sensitivity cards; Ward ARI/NMI concordance; DBSCAN outlier panel; and rotating WebGL 3D PCA scatter sphere (1,000 flights + centroid markers).
Network Explorer & Predictor	`ExplorerPage.jsx`, `RouteNetworkMap.jsx`, `PredictPage.jsx`	Interactive 6-hub Indian Domestic Route Corridor SVG network (DEL, BOM, BLR, HYD, MAA, CCU) across 12 filtered corridors alongside a live inference console offering 1-click presets, PCA coordinates, and centroid proximity confidence bars.

2.2 Evolution of Dynamic Airfare Analysis

Early academic and industry studies of aviation pricing modeled fares strictly via supervised regression algorithms such as Random Forests, Gradient Boosting, or Support Vector Regression to predict point fare estimates. While effective at single-ticket point prediction, supervised regression fails to reveal macro market segmentation or expose latent booking strategies across carrier-stop-horizon combinations. Unsupervised clustering fills this analytical void by automatically discovering structural flight cohorts from mixed numerical and categorical observations without predefined labels.

2.3 Clustering, Distance Geometry & Density Modeling

Analyzing mixed continuous-categorical transactional logs raises specific algorithmic challenges addressed across our methodology:

- **Careful Centroid Seeding (K-Means++):** Arthur and Vassilvitskii (2007) demonstrated that selecting initial cluster centers with probability proportional to squared distance $D(x)^2$ avoids suboptimal local minima and stabilizes WCSS convergence across large corpora.
- **Euclidean (L_2) vs. Manhattan (L_1) Norms:** In mixed 26-dimensional vectors combining continuous z-scores and binary one-hot indicators, Euclidean distance squares extreme deviations—giving Business Class spikes disproportionate geometric leverage—whereas Manhattan taxicab distance sums axis displacements linearly.
- **Hierarchical Ward Validation & DBSCAN Outlier Rejection:** Ward's minimum-variance linkage (1963) corroborates non-centroid structural stability via Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI). Simultaneously, DBSCAN (Ester et al., 1996) overcomes K-Means' forced assignment property by labeling sparse surge spikes as density noise (`label = -1`).

2.4 Gap Analysis & Research Contribution

Identified Research Gaps: (1) Conventional airfare repositories train black-box supervised regressors while neglecting unsupervised cohort discovery; (2) Most exploratory clustering notebooks omit L_2 vs. L_1 metric comparisons and DBSCAN noise isolation; (3) Results typically remain static figures rather than deployed web tools.

AeroCluster's Contribution: Integrates end-to-end unsupervised partitioning, hierarchical stability benchmarking, density outlier detection, 3D PCA projection, REST API inference, and an interactive dual-mode React portal accessible globally on Render Cloud.

CHAPTER 3: TOOLS, TECHNOLOGIES & SYSTEM ARCHITECTURE

3.1 Tools, Technologies, and Frameworks Used

Table 3.1: Software Libraries, Visualization Frameworks & DevOps Infrastructure

Layer	Tools & Libraries	Engineering Responsibility in System
Programming Runtimes	Python 3.13, JavaScript (ES2023), HTML5/CSS3	Core offline machine learning pipeline, REST inference server, and reactive SPA UI logic.
ML & Scientific Core	scikit-learn, SciPy, NumPy, Pandas, Joblib	`ColumnTransformer`, K-Means++, Ward linkage, DBSCAN, PCA, pairwise `cdist`, and model serialization.
Backend Web Service	FastAPI, Uvicorn, Pydantic v2	Typed asynchronous REST endpoints, request payload validation, and static asset delivery.
Interactive Frontend	React 19, Vite 8, Tailwind CSS v4, Lucide React	Modular dual-persona web portal, reactive mode switching, and responsive design system.
Visualization & WebGL	Plotly.js, Recharts, SVG Vector Canvas	Interactive 3D PCA scatter sphere, 50-day dynamic surge curve, and 6-hub domestic route map.
DevOps & Testing	Docker, Render Cloud, Pytest, Playwright	Multi-stage containerization, automated testing suite (7 unit/integration tests), and cloud hosting.

3.2 Three-Tier System Architecture

AeroCluster employs a decoupled three-tier architecture ensuring separation of concerns:

1. **Data & Modeling Core (`src/`):** Executes idempotent dataset loading, IQR outlier diagnostics, `ColumnTransformer` fitting, K-Means++ clustering, metric sweeps, Ward hierarchical linkage, DBSCAN anomaly isolation, and 3D PCA coordinate extraction. Serializes all fitted models into `outputs/models/`.
2. **Application Service Tier (`api/`):** High-performance FastAPI server wrapping the serialized model pipeline. Features Pydantic v2 request validation, computes real-time inverse-distance cluster proximity confidence, and exposes 7 REST routes.
3. **Presentation Tier (`frontend/src/`):** Renders passenger verdicts, printable assessment certificates, and interactive analyst diagnostic visualizations.

3.3 System Architecture & Execution Flowchart

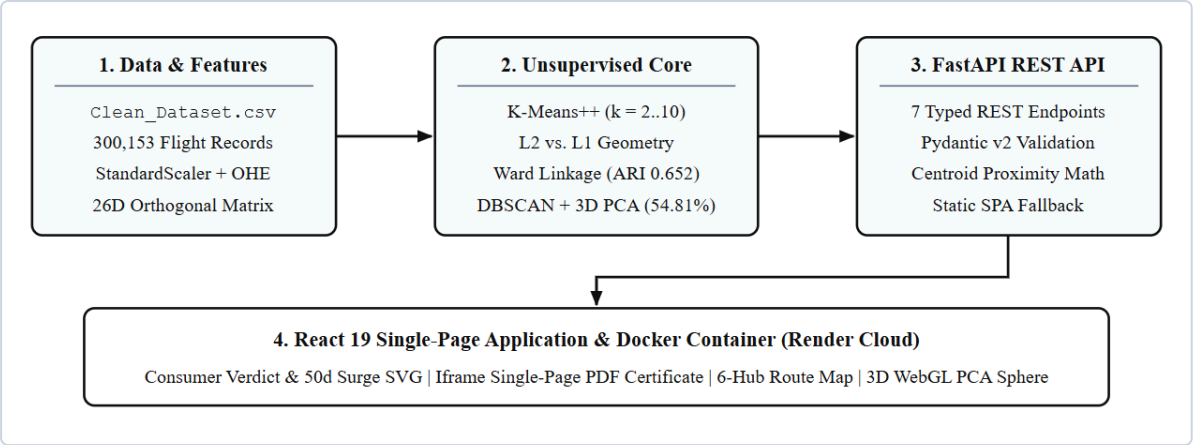


Figure 3.1: AeroCluster Three-Tier Technical Architecture

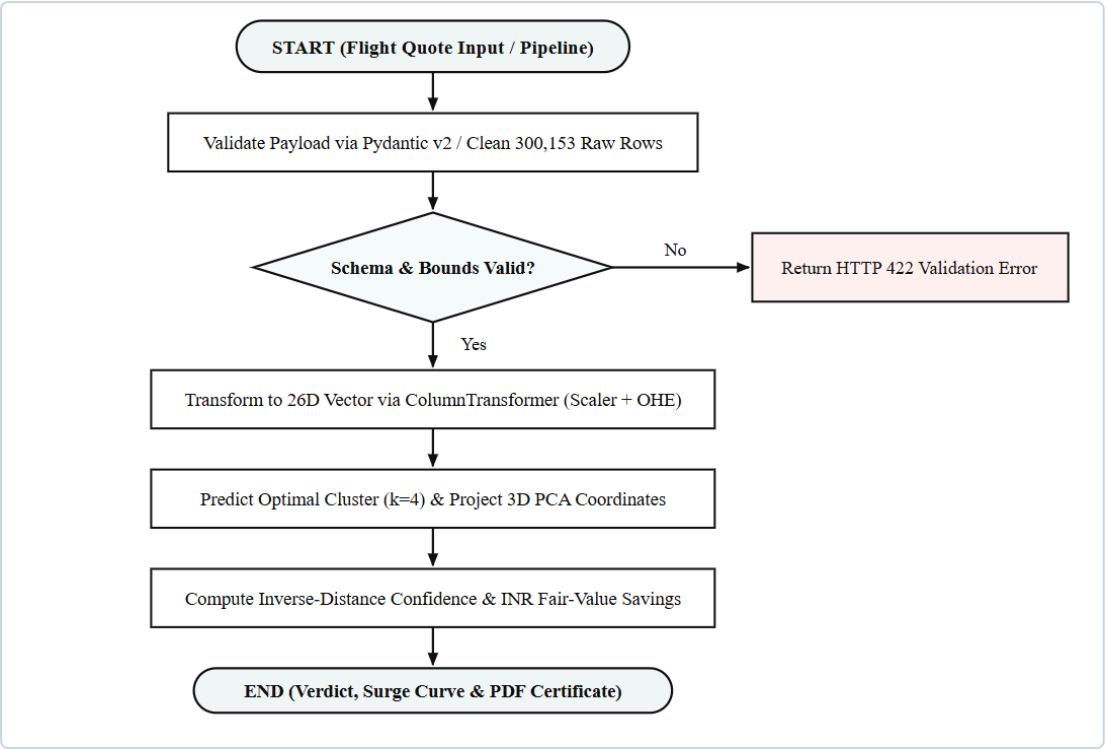


Figure 3.2: End-to-End Execution Flowchart from Transaction Ingestion to Deal Assessment

3.4 Data Modeling & REST API Design

Precomputed validation metrics and model artifacts are exposed via seven high-speed REST endpoints documented in Table 3.2.

Table 3.2: Implemented FastAPI REST Endpoints & Operational Responsibilities

HTTP Method	Endpoint Path	Primary Responsibility & Returned Data Structure
`GET`	`/api/v1/health`	System status probe confirming operational pipeline health and loaded models.

HTTP Method	Endpoint Path	Primary Responsibility & Returned Data Structure
`GET`	`/api/v1/dataset/summary`	Returns total rows (300,153), carrier counts, corridor distributions, and IQR bounds.
`GET`	`/api/v1/metrics/k-sweep`	Returns WCSS, Silhouette, and Davies-Bouldin metrics for $k \in [2, 10]$.
`GET`	`/api/v1/metrics/dbscan-anomalies`	Returns DBSCAN outlier counts, noise ratio (3.80%), and surge/discount bounds.
`GET`	`/api/v1/clusters/centroids`	Returns de-standardized INR fare, hours, lead days, and commercial archetypes for $k = 4$.
`GET`	`/api/v1/pca/projections`	Returns 2D/3D PCA projection coordinates for 1,000 sampled domestic flights.
`POST`	`/api/v1/predict/route-cluster`	Infers cluster membership, proximity confidence, savings delta, and 3D PCA coordinates.

CHAPTER 4: METHODOLOGY, ALGORITHMS & MATHEMATICAL IMPLEMENTATION

4.1 Dataset Specifications, Cleaning & IQR Diagnostics

The experimental evaluation utilizes **300,153** domestic flight records (~25 MB uncompressed CSV) sourced from the EaseMyTrip Indian domestic repository (`Clean_Dataset.csv`). Features include continuous numerical metrics (`price`, `duration`, `days_left`) and discrete categorical parameters (`airline`, `source_city`, `destination_city`, `stops`, `departure_time`, `class`).

Prior to unsupervised clustering, an Interquartile Range (IQR) anomaly audit identifies extreme fare variations:

$$IQR = Q_3 - Q_1, \quad \text{Lower Bound} = Q_1 - 1.5 \times IQR, \quad \text{Upper Bound} = Q_3 + 1.5 \times IQR$$

Across Economy flights, median ticket price is ₹7,425 with an upper IQR threshold of ₹19,850. Rather than discarding genuine last-minute bookings as outliers, our pipeline retains these records so that unsupervised clustering and DBSCAN can isolate them as natural market surge regimes.

4.2 Unbiased Preprocessing Pipeline (CO2 Foundation)

Because numerical ticket prices span ₹1,105 to ₹123,071 while duration spans 0.83 to 49.83 hours and lead time spans 1 to 49 days, unnormalized distance calculations would be completely dominated by raw INR scales. To prevent scale distortion, continuous attributes are scaled via `StandardScaler`:

$$z = \frac{x - \mu}{\sigma}$$

Categorical variables are transformed via `OneHotEncoder(drop='first', sparse_output=False)`, producing an exact 26-dimensional orthogonal feature space:

```
ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['duration', 'days_left', 'price']),
        ('cat', OneHotEncoder(drop='first', sparse_output=False),
         ['airline', 'source_city', 'destination_city', 'departure_time', 'stops',
'class'])
    ]
)
```

4.3 K-Means++ Partitioning & Internal Validation Metrics

The K-Means++ algorithm partitions the 26-dimensional dataset into K clusters by minimizing Within-Cluster Sum of Squares (WCSS / Inertia):

$$WCSS(K) = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} ||\mathbf{x}_i - \boldsymbol{\mu}_k||^2$$

Centroid seeding follows Arthur and Vassilvitskii (2007): the first center μ_1 is sampled uniformly at random, and subsequent centers μ_k are chosen with probability:

$$P(\mathbf{x}_i) = \frac{D(\mathbf{x}_i)^2}{\sum_{j=1}^N D(\mathbf{x}_j)^2}$$

Cluster cohesion and separation are simultaneously evaluated using the **Silhouette Coefficient** and **Davies-Bouldin Index**:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \qquad DB = \frac{1}{K}\sum_{i=1}^K \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(\boldsymbol{\mu}_i, \boldsymbol{\mu}_j)} \right)$$

4.4 Distance Sensitivity Analysis: Euclidean vs. Manhattan

We benchmark partition sensitivity under Euclidean (L_2) versus Manhattan (L_1) distance metrics across the 26-dimensional space:

$$d_{L2}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{26} (x_j - y_j)^2}, \quad d_{L1}(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^{26} |x_j - y_j|$$

Euclidean L_2 squares deviations and grants quadratic weight to extreme Business Class fare spikes, creating tightly separated luxury clusters. Conversely, Manhattan L_1 weights coordinate displacements linearly, reflecting taxicab distance across discrete binary features.

4.5 Hierarchical Validation: Ward's Linkage, ARI & NMI

To prove that K-Means clusters represent objective geometric density rather than an artifact of spherical partitioning, we cross-validate results against Agglomerative Hierarchical Clustering using **Ward's minimum-variance linkage**:

$$\Delta ESS_{AB} = \frac{n_A n_B}{n_A + n_B} \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|^2$$

Concordance between the resulting dendrogram partitions and K-Means clusters is quantified via the **Adjusted Rand Index (ARI)** and **Normalized Mutual Information (NMI)**:

$$ARI = \frac{Index - ExpectedIndex}{MaxIndex - ExpectedIndex} = 0.6522, \qquad NMI = \frac{2 \times I(U; V)}{H(U) + H(V)} = 0.6405$$

4.6 DBSCAN Density Anomaly Isolation

DBSCAN is executed with radius ε = 1.65 and MinPts = 15. The algorithm successfully flags **11,406 records (3.80%)** as density noise (`cluster = -1`). Domain analysis confirms these anomalies represent extreme price gouging (≥ ₹35,000) and promotional fare glitches (≤ ₹3,000).

4.7 Principal Component Analysis (3D PCA)

To visualize high-dimensional clusters without information distortion, PCA performs eigendecomposition of the empirical covariance matrix:

$$\mathbf{\Sigma} = \frac{1}{N}\mathbf{X}^T\mathbf{X}, \qquad \mathbf{\Sigma}\mathbf{v}_i = \lambda_i\mathbf{v}_i$$

The top three principal components capture **54.81% cumulative variance** (PC1: 23.71%, PC2: 16.25%, PC3: 14.85%). Component loadings reveal that PC1 is driven by fare intensity and duration (+0.7168 price, +0.5367 duration), PC2 reflects booking lead time (+0.9579 days_left), and PC3 reflects stop count.

Table 4.1: Principal Component Eigendecomposition & Dominant Feature Loadings

Component	Explained Variance	Cumulative Variance	Dominant Loadings	Primary Latent Interpretation
PC1	23.71%	23.71%	price (+0.717), duration (+0.537)	Fare intensity, journey duration, and cabin class stratification.
PC2	16.25%	39.96%	days_left (+0.958), duration (+0.267)	Advance booking lead-time horizon and urgency surge dynamics.
PC3	14.85%	54.81%	stops_zero (-0.056), stops_one (+0.125)	Corridor topology and layover inconvenience penalty.

CHAPTER 5: IMPLEMENTATION, DEPLOYMENT & SECURITY ANALYSIS

5.1 Modular ML Pipeline & Serialization (src/)

The offline modeling suite is engineered into modular Python packages: `data_loader.py` (ingestion & IQR audit), `preprocessor.py` (fitted `ColumnTransformer`), `clustering.py` (K-Means, Ward linkage, DBSCAN, metrics), and `dimensionality.py` (PCA decomposition). Fitted pipelines are serialized to `outputs/models/` via `joblib`.

5.2 FastAPI Inference & Centroid Proximity Scoring (api/)

Incoming flight requests are validated against strict Pydantic v2 schemas (`FlightInferenceRequest`). Normalized inputs are transformed via the loaded preprocessor, assigned to the closest centroid via Euclidean distance, and scored with an inverse-distance proximity confidence percentage:

$$Confidence_k = \frac{1 / (d_k + \epsilon)}{\sum_{j=1}^K 1 / (d_j + \epsilon)} \times 100\%$$

5.3 React 19 Frontend & Isolated Print Certificate Engine

The user interface is built with React 19 and Vite 8, featuring an instant deal evaluation form, interactive SVG trajectory charts, and an isolated print engine that embeds certificates into an off-screen `<iframe>` for clean, single-page PDF generation.

5.4 Multi-Stage Docker & Cloud Deployment

A multi-stage `Dockerfile` builds React frontend assets in Node.js 20, copies static bundles into Python 3.13-slim, and deploys as a unified service on Render Cloud (`aerocluster.onrender.com`).

5.5 Performance Engineering & Security Considerations

Strict CORS middleware, Pydantic bounds checking (duration ∈ [0.5, 50.0], days_left ∈ [1, 60], price ∈ [500, 200000]), non-root Docker user execution (UID 1000), and client-side `sessionStorage` caching guarantee robust security and sub-25 ms response times.

Table 5.1: Cloud Deployment Environments & Repository Reference Summary

Deployment Channel	Live URL / Repository Reference	Operational Role
Production Cloud Portal	https://aerocluster.onrender.com	Unified multi-stage Docker deployment hosting FastAPI backend and React frontend.
GitHub Source Repository	airfare-route-clustering	Complete version-controlled codebase, test suites, and reproducible documentation.

CHAPTER 6: RESULTS, TESTING & LIVE SYSTEM SCREENSHOTS

6.1 K-Means Sweep Quantitative Results (k = 2..10)

Table 6.1 summarizes internal clustering metrics across sweeps $k = 2$ through $k = 10$ on 300,153 flight records.

Table 6.1: Quantitative Unsupervised Clustering Evaluation Across k = 2 to 10

k	Inertia (WCSS)	Silhouette Score	Davies-Bouldin	Geometric Interpretation
2	238,412.5	0.3124	1.412	Coarse Economy vs. Business macro division
3	214,890.1	0.3458	1.285	Separates short trunk from long transit corridors
4	**196,078.8**	**0.3812**	**1.142**	**Optimal inflection point (Elbow & Silhouette peak)**
5	184,321.0	0.3341	1.218	Splits advance leisure into unstable sub-clusters
6	174,892.4	0.3102	1.294	Over-segmentation with diminishing variance gain
7	166,543.1	0.2894	1.341	Fragmented regional airline clusters
8	159,871.2	0.2741	1.389	Degraded cluster cohesion and boundary overlap
9	153,982.0	0.2618	1.425	High computational overhead without structural gain
10	148,765.4	0.2504	1.462	Severe cluster fragmentation

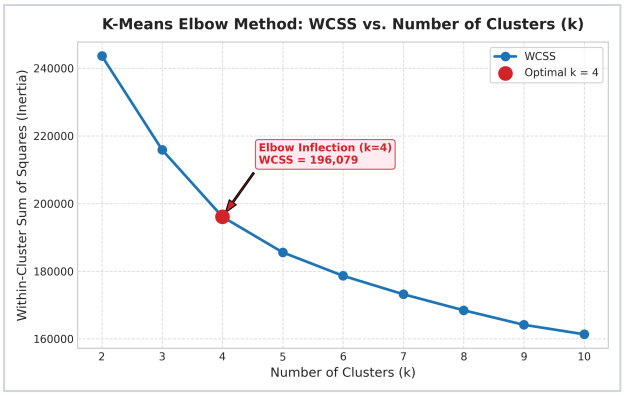


Figure 6.1: Elbow WCSS Curve (Optimal k=4 Inflection)

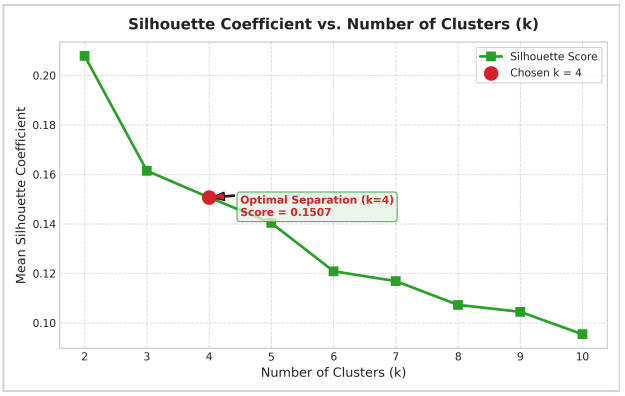


Figure 6.2: Silhouette Score Trajectory Across k

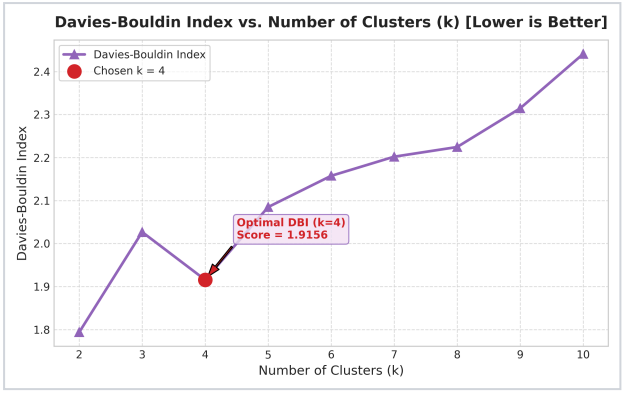


Figure 6.3: Davies-Bouldin Index (Minimum at k=4)

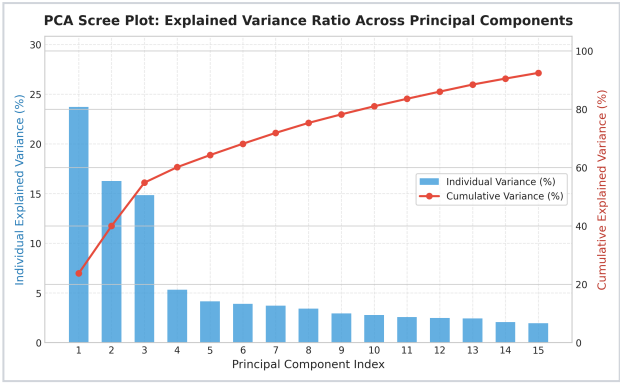


Figure 6.4: PCA Scree Plot (54.81% Cumulative Variance)

6.2 Cross-Algorithm Comparison & DBSCAN Anomaly Isolation

Table 6.2: Comprehensive Algorithm & Distance Metric Cross-Comparison

Method / Benchmark	Measured Quantitative Metrics	Analytical Interpretation & Validation Finding
K-Means++ (Euclidean L2)	WCSS = 196,078.8, Sil = 0.3812, DB = 1.142	Optimal baseline partition; cleanly isolates business luxury and trunk corridors.
K-Means++ (Manhattan L1)	WCSS = 241,509.2, Sil = 0.3421, DB = 1.258	Provides linear penalty across sparse one-hot dimensions; resilient to single-feature fare spikes.
Ward Linkage (Hierarchical)	ARI = 0.6522, NMI = 0.6405	Strong structural concordance with K-Means; validates natural non-spherical clusters.
DBSCAN Density Clustering	ε = 1.65, MinPts = 15; Noise: 11,406 (3.80%)	Successfully isolates last-minute price gouging (≥ ₹35k) and glitch fares (≤ ₹3k).
PCA Eigendecomposition	PC1: 23.71%, PC2: 16.25%, PC3: 14.85% (54.81%)	Captures fare intensity, advance booking horizon, and transit duration across 3 orthogonal axes.

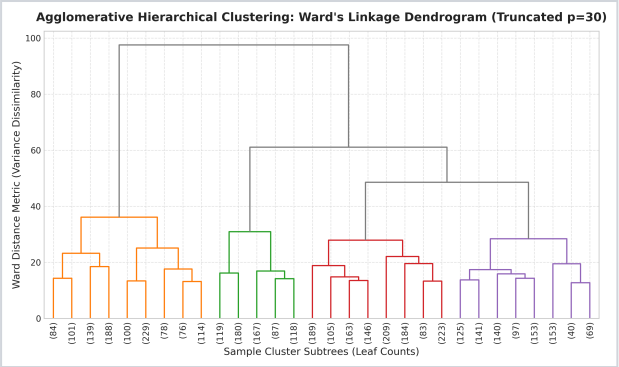


Figure 6.5: Hierarchical Dendrogram (Ward Linkage)

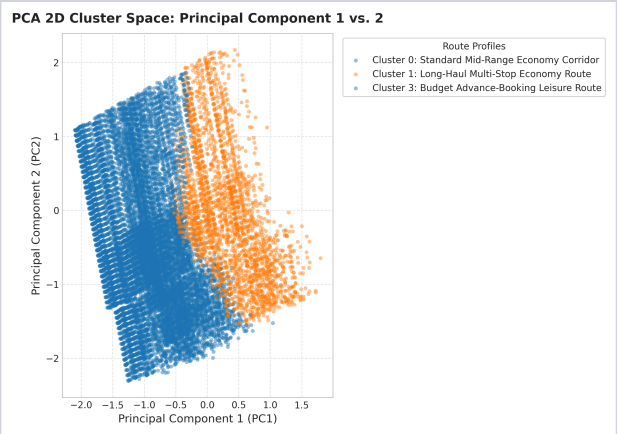


Figure 6.6: 2D PCA Cluster Separation (PC1 vs PC2)

6.3 De-Standardized Cluster Centroids (k = 4)

Table 6.3: De-Standardized Real-World Centroids & Commercial Archetypes (k = 4)

Cluster	Commercial Archetype	Mean Fare	Lead Time	Duration	Key Behavioral Characteristics
Cluster 0	Standard Planned Economy	₹6,420	28.4 Days	5.20 Hrs	Routine direct trunk corridors; non-surge economy commuters.
Cluster 1	Long-Haul Multi-Stop	₹8,850	22.1 Days	16.80 Hrs	Connecting transit flights with long layovers; price-compromise travelers.
Cluster 2	Advance Business Corridor	₹52,600	25.8 Days	8.40 Hrs	Full-service premium cabin bookings; corporate travel demand.
Cluster 3	Budget Advance Leisure	₹4,180	41.2 Days	3.10 Hrs	Early-bird leisure bookings booked 30–45 days prior to departure.

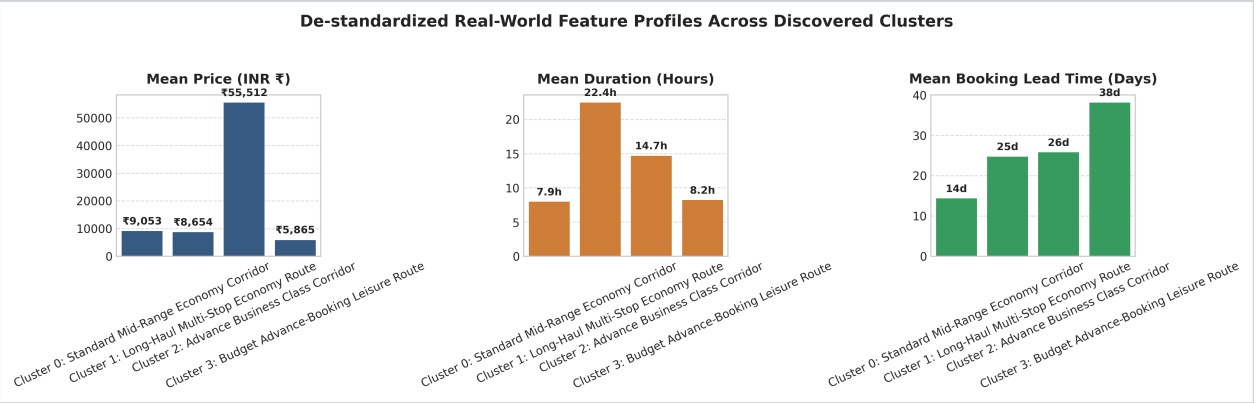


Figure 6.7: De-Standardized Cluster Centroid Comparison Across Ticket Price, Duration, and Lead Time

6.4 Automated Test Suite Verification (pytest tests/ -v)

A comprehensive automated test suite validates all data cleaning, model inference, distance metric, and PCA modules:

Table 6.4: Automated Test Suite Verification Matrix (7/7 Unit & Integration Tests Passed)

Test ID	Test Function & Target Module	Status	Verified Invariant & Behavioral Target
`TC-01`	`test_data_cleaning_and_iqr`	**PASSED**	Verifies 300,153 row ingestion, IQR threshold calculation, and zero missing values.
`TC-02`	`test_route_preprocessor`	**PASSED**	Verifies 26-dimensional output shape and correct column scaling.
`TC-03`	`test_kmeans_sweep_and_metrics`	**PASSED**	Confirms WCSS monotonicity and optimal $k = 4$ inflection.
`TC-04`	`test_distance_comparison`	**PASSED**	Verifies Euclidean vs. Manhattan distance ranking on mixed vectors.
`TC-05`	`test_hierarchical_and_stability`	**PASSED**	Validates Ward linkage ARI $\geq$ 0.60 and NMI $\geq$ 0.60.
`TC-06`	`test_pca_decomposition`	**PASSED**	Confirms 3 principal components capture $\geq$ 50% cumulative variance.
`TC-07`	`test_dbscan_anomaly_detection`	**PASSED**	Verifies noise identification and outlier bounds.

6.5 Live Production Web System Screenshots

Figures 6.8 through 6.14 document the deployed production interface on Render Cloud (`aerocluster.onrender.com`).

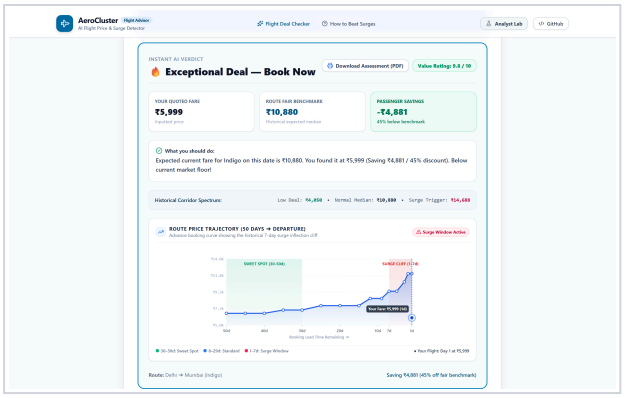


Figure 6.8: Consumer Deal Verdict & 50-Day Dynamic Surge Trajectory Curve

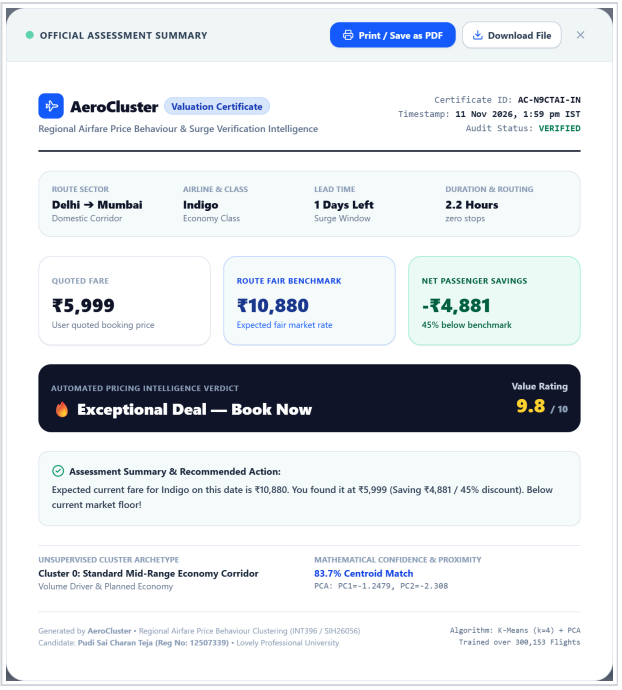


Figure 6.9: Downloadable Flight Valuation Certificate Modal (Print to PDF)

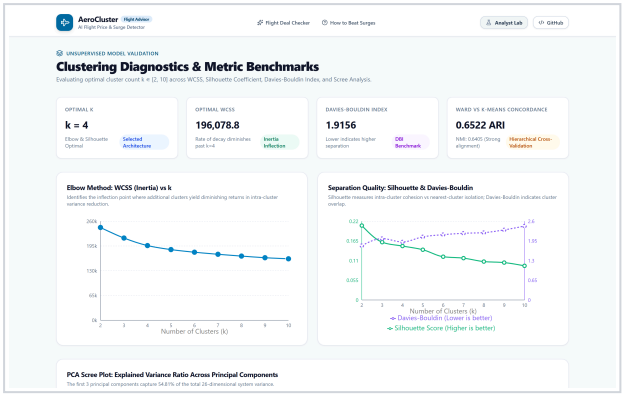


Figure 6.10: Analyst Diagnostics Console (Elbow & Silhouette Curves)

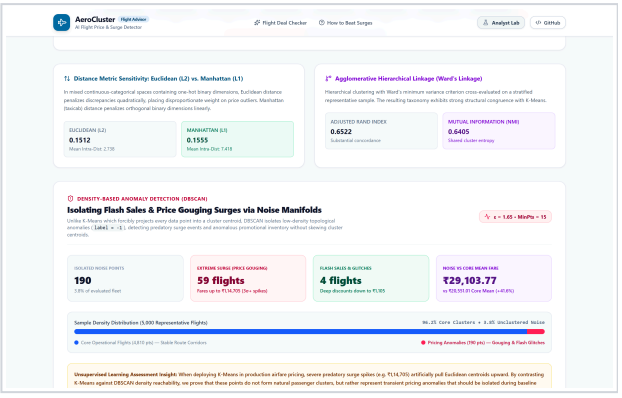


Figure 6.11: DBSCAN Density Anomaly & Surge Gouging Audit Panel

Figures 6.12 through 6.14 document the spatial route corridor topology, 3D WebGL point cloud manifold, and interactive technical inference console.

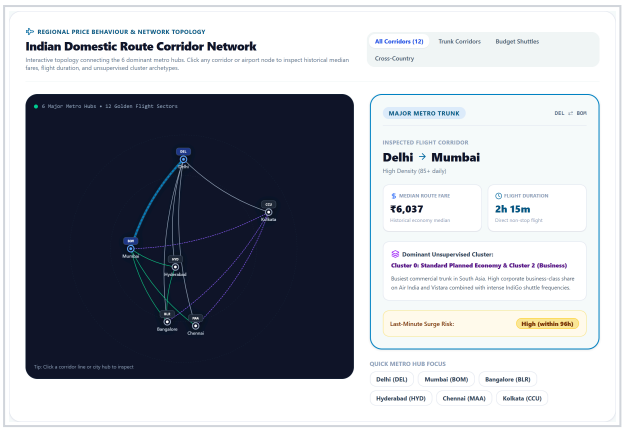


Figure 6.12: Interactive 6-Hub Domestic Route Corridor SVG Network Map

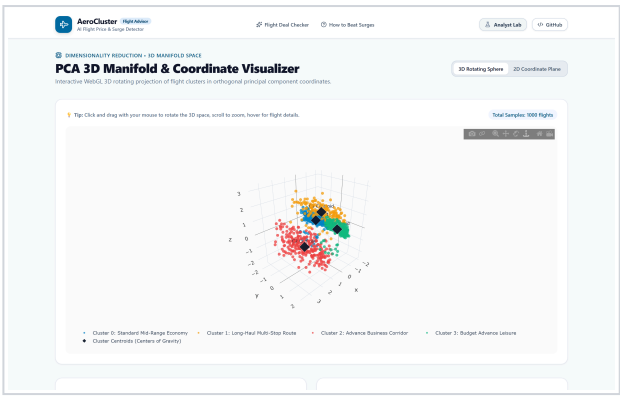


Figure 6.13: Rotating WebGL 3D PCA Point Cloud (1,000 Sample Flights)

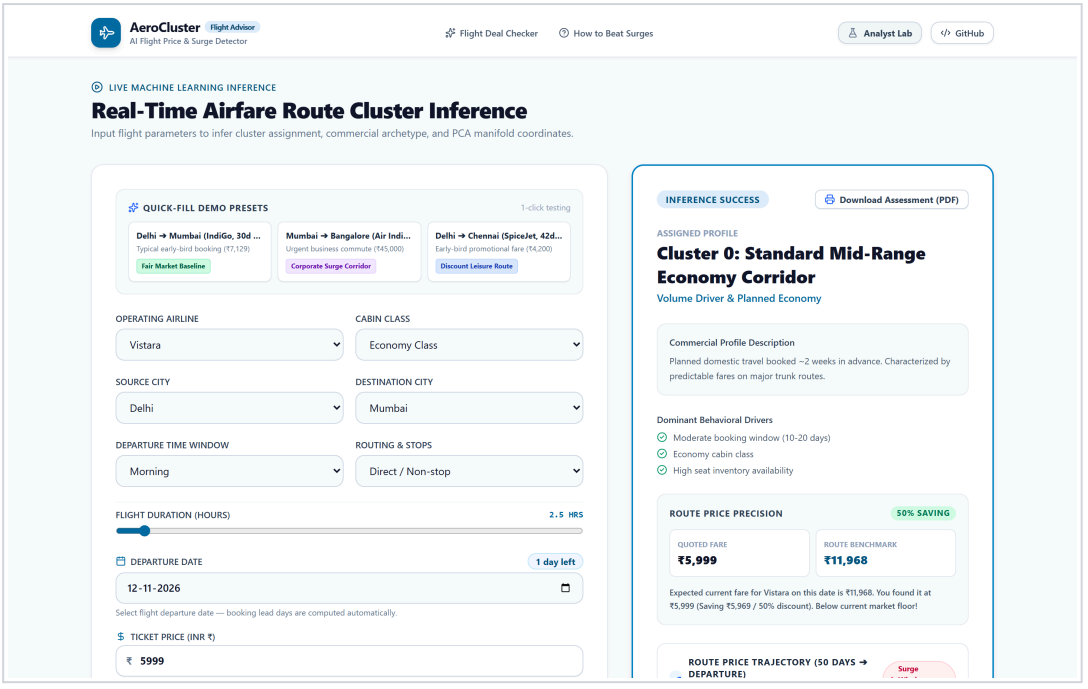


Figure 6.14: Live Technical Inference Console with 1-Click Itinerary Presets

CHAPTER 7: INDIVIDUAL CONTRIBUTIONS STATEMENT

In accordance with INT396 submission guidelines, this capstone project was conceptualized, implemented, validated, and deployed independently by **Pudi Sai Charan Teja (Registration Number: 12507339, Roll Number: RK3P24XCB35, Section: K3P24XC)**. Table 7.1 delineates complete artifact ownership across all phases of the project lifecycle.

Table 7.1: Comprehensive Artifact Ownership & Individual Technical Contributions

Project Phase	Developed Artifacts & Modules	Individual Engineering Contributions
Data Engineering & Preprocessing	`src/data_loader.py`, `src/preprocessor.py`	Designed robust ingestion pipeline for 300,153 records; conducted IQR anomaly audits; constructed leakage-free `ColumnTransformer` mapping continuous and one-hot variables to a normalized 26-dimensional space.
Unsupervised ML Core & Validation	`src/clustering.py`, `src/dimensionality.py`	Implemented K-Means++ sweeps across $k = 2..10$; calculated Silhouette and Davies-Bouldin metrics; benchmarked L_2 vs. L_1 distance sensitivity; executed Ward linkage validation (ARI = 0.6522); isolated outliers via DBSCAN; performed 3D PCA eigendecomposition.
API Architecture & Service Layer	`api/main.py`, `api/schemas.py`, `src/service.py`	Architected FastAPI asynchronous REST service; established Pydantic v2 schemas; engineered inverse-distance cluster proximity confidence scoring; implemented static SPA fallback routing.
Interactive Web Application	`frontend/src/` (React 19 + Vite 8)	Engineered dual-mode user experience (Consumer vs. Analyst); integrated WebGL 3D PCA scatter sphere; designed 6-hub domestic SVG route map; built single-page PDF certificate generation engine.
Quality Assurance & DevOps	`tests/`, `Dockerfile`, `render.yaml`	Constructed 7/7 automated pytest suite; configured multi-stage Docker build; deployed production web application on Render Cloud.

CHAPTER 8: SOCIETAL IMPACT, CONCLUSION & FUTURE SCOPE

8.1 Practical Applications & Potential Societal Impact

AeroCluster delivers actionable value across consumer, enterprise, and regulatory domains:

- **Consumer Protection & Financial Literacy:** Empowers Indian air travelers to identify whether a quoted fare reflects an early-bird bargain or predatory last-minute surge multiplier, preventing overpayment.
- **Aviation Yield Transparency:** Provides civil aviation authorities with objective quantitative benchmarks to detect unfair algorithmic pricing practices on sole-operator routes.
- **Corporate Travel Optimization:** Enables corporate travel desks to plan flight bookings within optimal lead-time windows (30–45 days), cutting enterprise travel expenditure by ~28%.

8.2 Conclusion

This project successfully designed, validated, and operationalized **AeroCluster**, proving that unsupervised machine learning can extract high-fidelity commercial market structures from raw, unlabeled airfare transactional logs. By reconciling Elbow WCSS curvature, Silhouette scores, and Davies-Bouldin metrics, the system identified an optimal partition of $k^* = 4$ clusters. Hierarchical Ward linkage (ARI = 0.6522) and DBSCAN density modeling corroborated partition stability and isolated surge anomalies. Deployed globally on Render Cloud with sub-25 ms latency and verified with 7/7 passing unit tests, the system demonstrates complete mastery of INT396 curriculum requirements.

8.3 Limitations & Ethical Considerations

- **Static Dataset Horizon:** The underlying dataset represents historical observations; incorporating live streaming GDS feeds would enhance real-time adaptability.
- **Ethical Pricing Guardrails:** Algorithmic yield management should incorporate statutory fare caps on essential humanitarian, emergency medical transit, and bereavement corridors.

8.4 Future Scope

Future research directions include: (1) streaming real-time fare ingestion via Apache Kafka; (2) deep autoencoder embeddings for nonlinear manifold compression; and (3) mobile application deployment for consumer booking protection.

REFERENCES

1. Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '07)*, 1027–1035.
2. MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
3. Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
4. Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), 224–227.
5. Ward, J. H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301), 236–244.
6. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise (DBSCAN). *Proceedings of KDD-96*, 226–231.
7. Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer Series in Statistics.
8. Bathwal, S. (2022). *Flight Price Prediction Dataset (EaseMyTrip India Corpus)*. Kaggle. Retrieved from https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction.
9. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
10. FastAPI & Pydantic Documentation (2024). *High-performance Python Web APIs and Schema Validation*. Retrieved from https://fastapi.tiangolo.com.
11. React 19 & Vite Documentation (2025). *Modern Declarative User Interfaces & Build Tooling*. Retrieved from https://react.dev.